

MIPS arrays and functions

1521 tutorial 4





00

Assignment brief

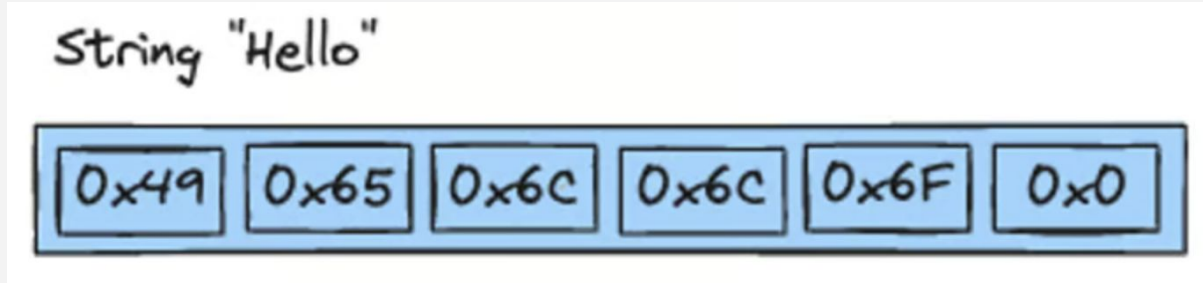


01

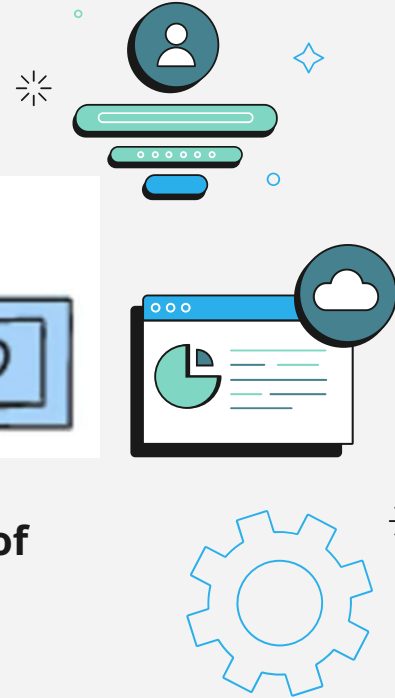
2D Arrays

Arrays

- 1D arrays in mips are just each array element consecutively in a straight line



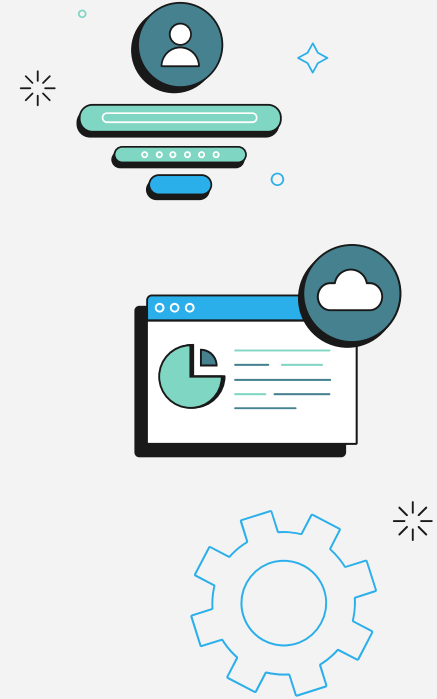
If we want to find element i , we multiply i by the size of the variables in the array and add that amount to the start of the array.



2D Arrays

- 2D arrays are just each row put after each other consecutively - still in a straight line!

2D array?					
	0	1	2	3	< col
0	a	b	c	d	
1	e	f	g	h	
2	i	j	k	l	
^ row					



2D Arrays

- To access element (0, i), we add i to the start address of the array.
- To access element (1, i), we add $i + 1 * (\text{amount of cols})$ to the start address of the array.
- To access element (2, i), we add $i + 2 * (\text{amount of cols})$ to the start address of the array.....

2D array?

	0	1	2	3	<	col
0	a	b	c	d		
1	e	f	g	h		
2	i	j	k	l		

^ row

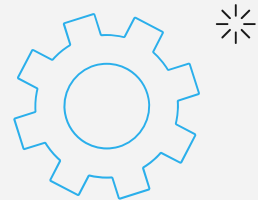
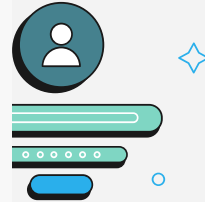
C TO MIPS - 2D Arrays

```
#include <stdio.h>

#define FLAG_ROWS 6
#define FLAG_COLS 12

char flag[FLAG_ROWS][FLAG_COLS] = {
    {'#', '#', '#', '#', '#', '.', '.', '#', '#', '#', '#', '#'},
    {'#', '#', '#', '#', '#', '.', '.', '#', '#', '#', '#', '#'},
    {'.', '.', '.', '.', '.', '.', '.', '.', '.', '.', '.', '.'},
    {'.', '.', '.', '.', '.', '.', '.', '.', '.', '.', '.', '.'},
    {'#', '#', '#', '#', '#', '.', '.', '#', '#', '#', '#', '#'},
    {'#', '#', '#', '#', '#', '.', '.', '#', '#', '#', '#', '#'}
};

int main(void) {
    for (int row = 0; row < FLAG_ROWS; row++) {
        for (int col = 0; col < FLAG_COLS; col++) {
            printf("%c", flag[row][col]);
        }
        printf("\n");
    }
}
```



C

```

# - row+col offset in $t3
main:
main__row_loop_init:                # for (int row = 0; row < FLAG_ROWS; row++) {
    li    $t0, 0                    # row = 0

    main__row_loop_body:            # while (row < FLAG_ROWS)
    bge    $t0, FLAG_ROWS, main__row_loop_end

    main__col_loop_init:            # for (int col = 0; col < FLAG_COLS; col++) {
    li    $t1, 0                    # col = 0;

    main__col_loop_body:            # while (col < FLAG_COLS)
    bge    $t1, FLAG_COLS, main__col_loop_end

                                # convert [row][col] to byte offset
    mul    $t2, $t0, FLAG_COLS      # 1) row offset = rowNumber * NUM_COLUMNS
    add    $t3, $t2, $t1            # 2) row+col offset = rowOffset + colNumber
                                # 3) byte offset = row+col offset * sizeof(char)
                                # (Step 3 is skipped since sizeof(char) == 1)
                                # 4) Add the base address of the flag array to the
    la     $t4, flag                # byte offset.
    add    $t5, $t3, $t4            # This produces the complete memory address of
    the flag element.

    lb     $a0, ($t5)               # 5) Load the byte at the byte offset into $a0

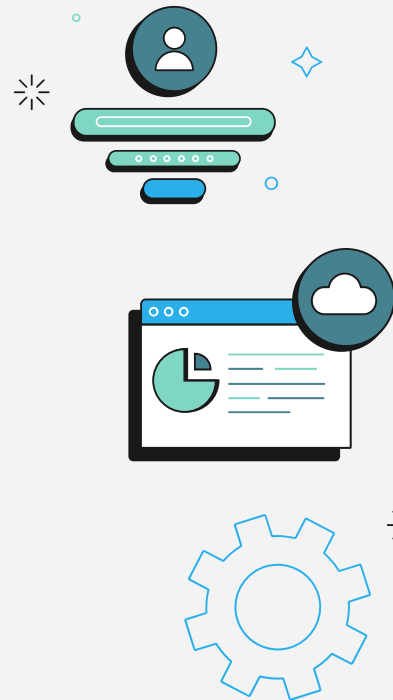
    li     $v0, 11                  # printf("%c", flag[row][col])
    syscall

    main__col_loop_iter:
    addi   $t1, $t1, 1              # col++
    j      main__col_loop_body

    main__col_loop_end:
    main__row_loop_iter:
    li     $a0, '\n'                # printf("%c", '\n')
    li     $v0, 11
    syscall
    addi   $t0, $t0, 1              # row++
    j      main__row_loop_body

    main__row_loop_end:
                                # End of the program.
    li     $v0, 0                    # return 0
    jr     $ra

```

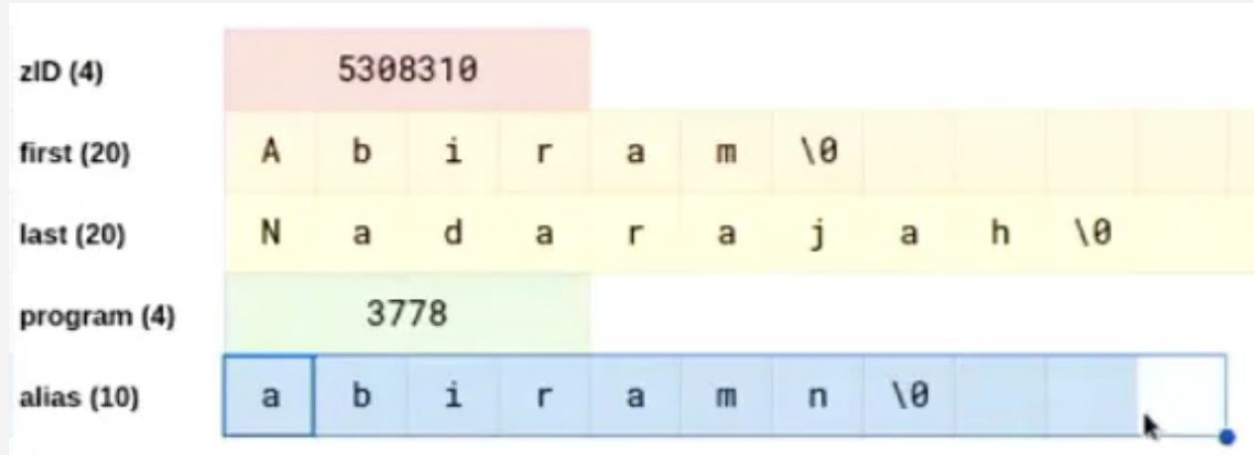
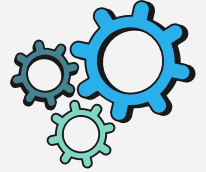
F
t
t
C
E



02

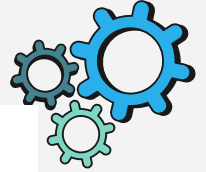
Structs

Structs

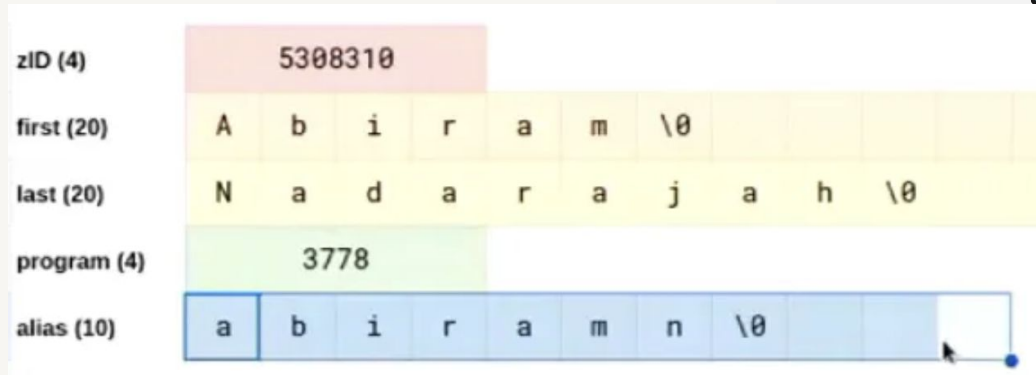


Structs are also laid out in more or less a straight line - with the exception that we need to care about spacing between variable types.

Structs



```
abiram:  
  .word    5308310  
  .asciiz  "abiram"  
  .space   20 - 7  
  .asciiz  "Nadarajah"  
  .space   20 - 10  
  .word    3778  
  .asciiz  "abiramn"  
  .space   10 - 2
```



Note that we can also use .space to reserve space in an array that hasn't been occupied by something

For each of the following struct definitions, what are the likely offset values for each field, and the total size of the struct ?:

a.

```
struct _coord {  
    double x;  
    double y;  
};
```

b.

```
typedef struct _node Node;  
struct _node {  
    int value;  
    Node *next;  
};
```

c.

```
struct _enrolment {  
    int stu_id;           // e.g. 5012345  
    char course[9]:      // e.g. "COMP1521"  
    char term[5];        // e.g. "17s2"  
    char grade[3];       // e.g. "HD"  
    double mark;         // e.g. 87.3  
};
```

d.

```
struct _queue {  
    int nitens;           // # items currently in queue  
    int head;            // index of oldest item added  
    int tail;            // index of most recent item added  
    int maxitems;        // size of array  
    Item *items;         // malloc'd array of Items  
};
```

- a) **x is the first field, so has offset 0. Since x is 8 bytes long, y will have offset 8. The struct holds two 8-byte fields, both of which are correctly aligned, so no padding is required, and so the overall size of the struct is 16 bytes.**

a.

```
struct _coord {  
    double x;  
    double y;  
};
```

b.

```
typedef struct _node Node;  
struct _node {  
    int value;  
    Node *next;  
};
```

c.

```
struct _enrolment {  
    int stu_id;           // e.g. 5012345  
    char course[9]:      // e.g. "COMP1521"  
    char term[5];        // e.g. "17s2"  
    char grade[3];       // e.g. "HD"  
    double mark;         // e.g. 87.3  
};
```

d.

```
struct _queue {  
    int nitems;           // # items currently in queue  
    int head;            // index of oldest item added  
    int tail;            // index of most recent item added  
    int maxitems;        // size of array  
    Item *items;         // malloc'd array of Items  
};
```

b) value is the first field, so has offset 0. Since value is 4 bytes long, next will have offset 4. Since pointers are also 4 bytes long, and no padding is required, the overall size of the struct is 8 bytes.

a.

```
struct _coord {  
    double x;  
    double y;  
};
```

b.

```
typedef struct _node Node;  
struct _node {  
    int value;  
    Node *next;  
};
```

c.

```
struct _enrolment {  
    int stu_id;           // e.g. 5012345  
    char course[9]:      // e.g. "COMP1521"  
    char term[5];        // e.g. "17s2"  
    char grade[3];       // e.g. "HD"  
    double mark;         // e.g. 87.3  
};
```

d.

```
struct _queue {  
    int nitens;           // # items currently in queue  
    int head;            // index of oldest item added  
    int tail;            // index of most recent item added  
    int maxitems;        // size of array  
    Item *items;         // malloc'd array of Items  
};
```

c) `stu_id` is the first field, so has offset 0. Since `stu_id` is 4 bytes long, `course` has offset 4. `course` is 9 bytes long, but since the following field only needs byte alignment, `term` has offset 13. Similarly, `grade` has offset 18. The `mark` field requires word alignment, and so it needs some padding to move the offset from 21 to 24. Taking into account that double values are stored in 8 bytes, the overall size of the struct is 32 bytes.

a.

```
struct _coord {  
    double x;  
    double y;  
};
```

b.

```
typedef struct _node Node;  
struct _node {  
    int value;  
    Node *next;  
};
```

c.

```
struct _enrolment {  
    int stu_id;           // e.g. 5012345  
    char course[9];      // e.g. "COMP1521"  
    char term[5];        // e.g. "17s2"  
    char grade[3];       // e.g. "HD"  
    double mark;         // e.g. 87.3  
};
```

d.

```
struct _queue {  
    int nitems;           // # items currently in queue  
    int head;            // index of oldest item added  
    int tail;            // index of most recent item added  
    int maxitems;        // size of array  
    Item *items;         // malloc'd array of Items  
};
```

d) nitems is the first field, so has offset 0. head has offset 4, tail has offset 8, maxitems has offset 12, and items has offset 16. It doesn't matter what the size of an Item is, or how any Items there are, since they are stored outside the struct. Since the pointer is 4 bytes long, and all the values are word-aligned, the overall size of the struct is 20 bytes.

a.

```
struct _coord {  
    double x;  
    double y;  
};
```

b.

```
typedef struct _node Node;  
struct _node {  
    int value;  
    Node *next;  
};
```

c.

```
struct _enrolment {  
    int stu_id;           // e.g. 5012345  
    char course[9];      // e.g. "COMP1521"  
    char term[5];        // e.g. "17s2"  
    char grade[3];       // e.g. "HD"  
    double mark;         // e.g. 87.3  
};
```

d.

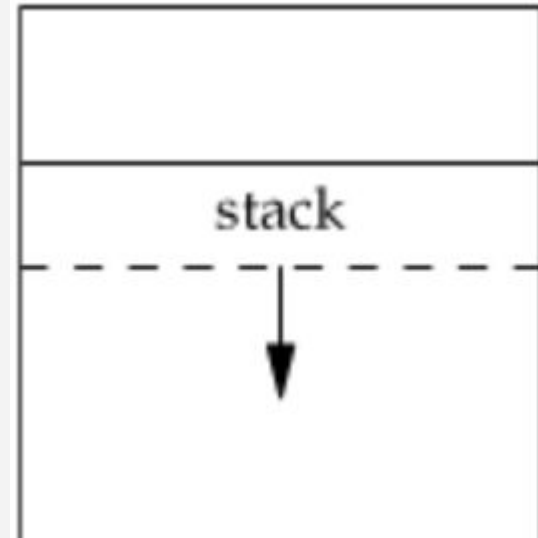
```
struct _queue {  
    int nitems;           // # items currently in queue  
    int head;            // index of oldest item added  
    int tail;            // index of most recent item added  
    int maxitems;        // size of array  
    Item *items;         // malloc'd array of Items  
};
```




03 MIPS functions

How do functions work in C?

```
int main(void) {  
    // function inside function  
    int num = 3;  
    print_num(num);  
}
```



How do functions work in MIPS?

- Functions in MIPS are just a label that defines where the function actually starts.
- We run a function by jumping to the label for that function
- This is why wacky things like function pointers exist - functions themselves are just addresses...

```
int main(void) {  
    // function inside function  
    int num = 3;  
    print_num(num);  
}
```

✓ J

Address₂₆

PC = PC[31-28] && Address₂₆ << 2

How do functions work in MIPS?

- One problem - if we're running a function that calls another function, how do we go back into the outer function when the inner function finishes running?

How do functions work in MIPS?

- One problem - if we're running a function that calls another function, how do we go back into the outer function when the inner function finishes running?



JAL

Address₂₆

JAL (Jump and link) sets the return address of the inner function to be the next line after the function call in the outer function. This means when we do `jr $ra` in the inner function, we end up in the outer function again.

One last thing...

- The \$t registers get **clobbered** whenever we use a JAL instruction
- The computer *will not remember* what was in them when we get back to the outer function.
- We prevent this using the \$s (stack) registers and two pseudo-instructions...

Writing functions in MIPS

```
// sum 4 numbers using function calls

#include <stdio.h>

int sum4(int a, int b, int c, int d);
int sum2(int x, int y);

int main(void) {
    int result = sum4(11, 13, 17, 19);
    printf("%d\n", result);
    return 0;
}

int sum4(int a, int b, int c, int d) {
    int res1 = sum2(a, b);
    int res2 = sum2(c, d);
    return sum2 (res1, res2);
}

int sum2(int x, int y) {
    return x + y;
}
```

```

main:
    push    $ra          # move stack pointer down to make room
                        # save $ra on $stack

    li      $a0, 11      # sum4(11, 13, 17, 19)
    li      $a1, 13
    li      $a2, 17
    li      $a3, 19
    jal     sum4

    move     $a0, $v0     # printf("%d", z);
    li      $v0, 1
    syscall

    li      $a0, '\n'    # printf("%c", '\n');
    li      $v0, 11
    syscall

    pop     $ra          # recover $ra from $stack
                        # move stack pointer back up to what it was when main
                        # equivalent to `pop $ra`

    li      $v0, 0       # return 0 from function main
    jr      $ra          # return from function main

```

```

sum4:
    # Prologue:
    # move stack pointer down to make room
    push    $ra          # save $ra on $stack
    push    $s0           # save $s0 on $stack
    push    $s2           # save $s2 on $stack
    push    $s3           # save $s3 on $stack
                        # We need to get the order out right as well!

sum4_body:
    move     $s2, $a2     # save $a2 in $s2
    move     $s3, $a3     # save $a3 in $s3

    jal     sum2          # sum2(a, b)
    move     $s0, $v0     # move result to $s0 to save it for Later

    # Move the preserved values back into the $a registers
    move     $s2, $a0     # restore $a2 from $s2
    move     $s3, $a1     # restore $a3 from $s3

    jal     sum2          # sum2(c, d);

    move     $a0, $s0     # Sum the first and second results
    move     $a1, $v0
    jal     sum2          # sum2(res1, res2)

    lw      $ra, 12($sp)  # restore $ra from $stack
    lw      $s0, 8($sp)   # restore $s0 on $stack
    addi    $sp, $sp, 16  # move stack pointer back up to what it was when sum4
                        # called

sum4_epilogue:
    pop     $s3           # pop from the stack into $s3
    pop     $s2           # pop from the stack into $s2
    pop     $s0           # pop from the stack into $s0
    pop     $ra           # pop from the stack into $ra

    jr      $ra          # return from sum_product

```

```

sum2:
    # sum2 doesn't call other functions,
    # so it doesn't need to save any registers.

    add     $v0, $a0, $a1 # return argument + argument 2
    jr      $ra          #

```


What should be put in each part of the function header comment?

Is it good style to use the function name as a prefix when naming labels?

```
max:
  # Frame:      [...]  <-- FILL THESE OUT!
  # Uses:       [...]
  # Clobbers:   [...]
  #
  # Locals:     <-- FILL THIS OUT!
  #   - ...
  #
  # Structure:  <-- FILL THIS OUT!
  #   max
  #   -> [prologue]
  #       -> body
  #   -> [epilogue]
max__prologue:

max__body:

max__epilogue:
```

copy_map:

```
# Subset: 2
#
# Args:
#   - $a0: char dst[MAP_HEIGHT][MAP_WIDTH]
#   - $a1: char src[MAP_HEIGHT][MAP_WIDTH]
#
# Returns: void
#
# Frame:  [$ra]
# Uses:   [$t0, $t1, $t2, $t3, $t4, $t5, $a0, $a1, $ra]
# Clobbers: [$t0, $t1, $t2, $t3, $t4, $t5, $a0, $a1]
#
# Locals:
#   - $t0: int i
#   - $t1: int j
#   - $t2: &dest
#   - $t3: &source
#   - $t4: load + save bytes
#   - $t5: multiplying array offset
#   - $a0: char dst[MAP_HEIGHT][MAP_WIDTH]
#   - $a1: char src[MAP_HEIGHT][MAP_WIDTH]
#
# Structure:
#   copy_map
#   -> [prologue]
#   -> body
#       -> copy map outer loop
#       -> copy map inner loop
#   -> [epilogue]

# copy_map copies the map given in the source array to the destination array.
```

MIPS... recursion????!

```
// C function to find the largest element in an array, recursively.
// Returns the value of the largest element in the array.
//
// array: Array to search
// length: Number of elements in the array
int max(int array[], int length) {
    //
    int first_element = array[0];
    if (length == 1) {
        // Handle the base-case of the recursion, at the end of the array.
        return first_element;
    } else {
        // Recurse on the rest of the array.
        // Finds the largest element after first_element in the array.
        int max_so_far = max(&array[1], length - 1);

        // Compare this element with the largest element after it in the array.
        if (first_element > max_so_far) {
            max_so_far = first_element;
        }
        return max_so_far;
    }
}
```

```

max__prologue:
    begin
    push    $ra
    push    $s0                # push $s0 to the stack to save its original value

max__body:
    lw      $s0, ($a0)         # load the value of array[0] into $s0

    bne     $a1, 1, max__length_gt_1    # if (length != 1), go to the recursive
case

max__base_case:                # base case of recursion
    move    $v0, $s0           # return first_element
    b       max__epilogue

max__length_gt_1:              # recursive case
    addi    $a0, $a0, 4         # Get the address of array[1] by adding 4 to array
                                # Works because each element is 4 bytes in size
    addi    $a1, $a1, -1        # length = length - 1;
    jal     max                 # recursive call:
    move    $t0, $v0            # max_so_far = max(&array[1], length - 1);

    ble     $s0, $t0, max__ret_max_so_far # if (first_element <= max_so_far), goto
max__ret_max_so_far
    move    $t0, $s0           # max_so_far = first_element

max__ret_max_so_far:
    move    $v0, $t0           # return value is max_so_far

max__epilogue:
    pop     $s0                # pop $s0 from the stack
    pop     $ra                # pop $ra from the stack
    end

    jr      $ra                # return;

```